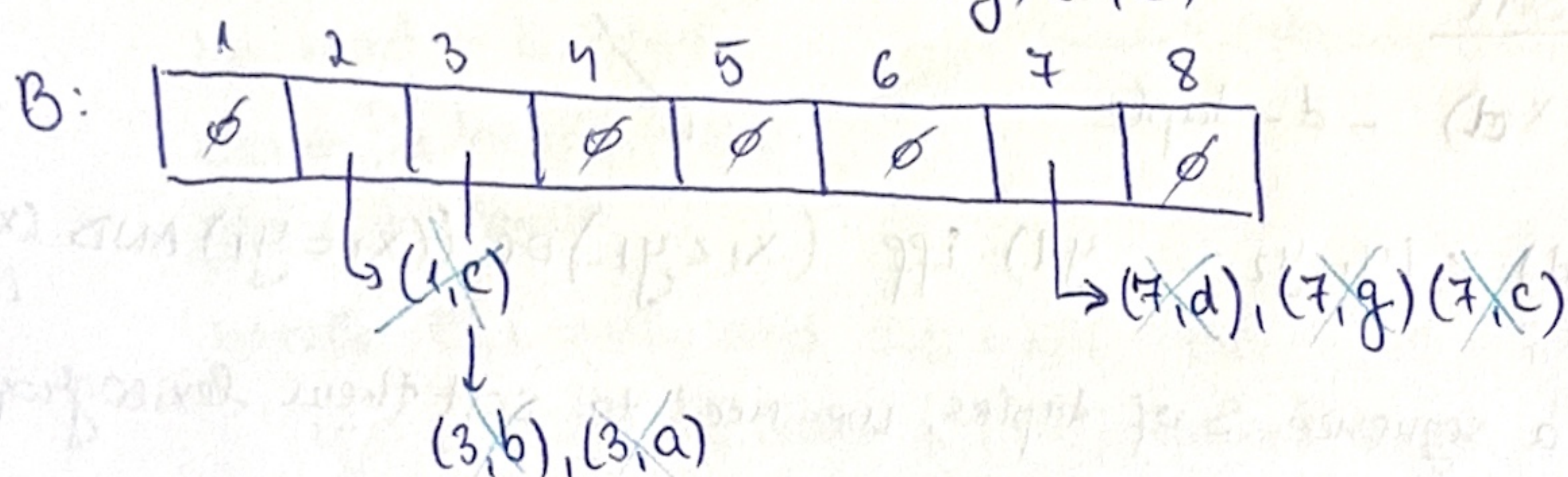


1. Bucket Sort $\rightarrow$ when the interval is small; when keys are integer numbers.

We are given a seq. of S , formed of m (key-value) pairs. The keys are integer numbers from $[0, N]$. Sort S based on the keys.

$S: (7, d), (1, c), (3, b), (7, g), (3, a), (7, e) \Rightarrow N=8$
 $\Rightarrow (1, c), (3, b), (3, a), (7, d), (7, g), (7, e)$



$S: (1, c), (3, b), (3, a), (7, d), (7, g), (7, e)$

ADT Sequence

$\Theta(1)$ {

- init(s) ✓
- removeFirst(s) ✓
- addToEnd(s, (k, v)) ✓
- destroy(s) ✓
- isEmpty(s) $\rightarrow$ bool ✓

} ~ Queue

• Circular Array has all operations $\Theta(1)$
 • Doubly / Singly Linked List

Subalgorithm BucketSort(S, N)

@ define array B with N empty seg.

while not isEmpty(s) execute:

$(K, v) \leftarrow \text{removeFirst}(s)$

$\text{addToEnd}(B[K], (K, v))$

□

for $i=0, N$ execute:

while not isEmpty($B[i]$) execute:

$(K, v) \leftarrow \text{removeFirst}(B[i])$

$\text{addToEnd}(S, (K, v))$

□

□

□

1) Create B

2) Empty seg.

3) loop through

$\Theta(m)$
 $\downarrow$
 nr. of pairs

$\Theta(m)$

$\Theta(m + N)$

(the maximum between m, N)

Obs! Keys need to be integer numbers.

STABLE sorting algorithm (if I have ~~exp~~ equal keys, the elems will be at in the order in which they were added):

ex: $(3, b) (3, a)$.

Bucket Sort is a STABLE sorting alg.

2. Lexicographic Sort

$(x_1, x_2, \dots, x_d)$ - d -tuple

this is how you compare d -tuples
String

$(x_1, x_2, \dots, x_d) < (y_1, y_2, \dots, y_d)$ iff $(x_1 < y_1)$ or $((x_1 = y_1) \text{ AND } (x_2, \dots, x_d) < (y_2, \dots, y_d))$

We are given a sequence S of tuples, we need to sort them lexicographically.
We will use:

- $R_i \rightarrow$ a relation that can compare two tuples, considering only dimension i .
- $\text{Stable Sort}(S, R) \rightarrow$ stable sorting algorithm which uses a relation.

subalgorithm Lexicographic Sort (S, d, R)

for $i \leftarrow d, 1, -1$, execute:

$\text{Stable Sort}(S, R_i);$

$\Theta(d \cdot T(n))$

complexity of stable s

ex: $d=3$

$S : (7, 4, 6) (5, 1, 5) (2, 4, 6) (2, 1, 4) (3, 2, 4)$

$d=3$ $(2, 1, 4), (3, 2, 4), (5, 1, 5), (7, 4, 6), (2, 4, 6)$

$d=2$ $(2, 1, 4) (5, 1, 5) (3, 2, 4) (7, 4, 6) (2, 4, 6)$

$d=1$ $(2, 1, 4) (2, 4, 6) (3, 2, 4) (5, 1, 5) (7, 4, 6)$

Implementation:

subalgorithm merge (L_1, L_2, LR): *check for empty lists!*
if not $L_1.R ([L_1.head].value, [L_2.head].value)$ then
 $tup \leftarrow L_1.head$
 $L_1.head \leftarrow L_2.head$
 $L_2.head \leftarrow tup$

ensure the lists contain the same relation

$LR.head \leftarrow L_1.head$
 $CL_1 \leftarrow [L_1.head].next$
 $CL_2 \leftarrow L_2.head$
current. (already added the head).

while $CL_2 \neq Nil$ and $CL_1 \neq Nil$ ex: *returns true if they're in the right order inside the parenthesis*

if $L_1.R ([CL_1].value, [CL_2].value)$ then

$chosen \leftarrow CL_1$
 $CL_1 \leftarrow [CL_1].next$

else

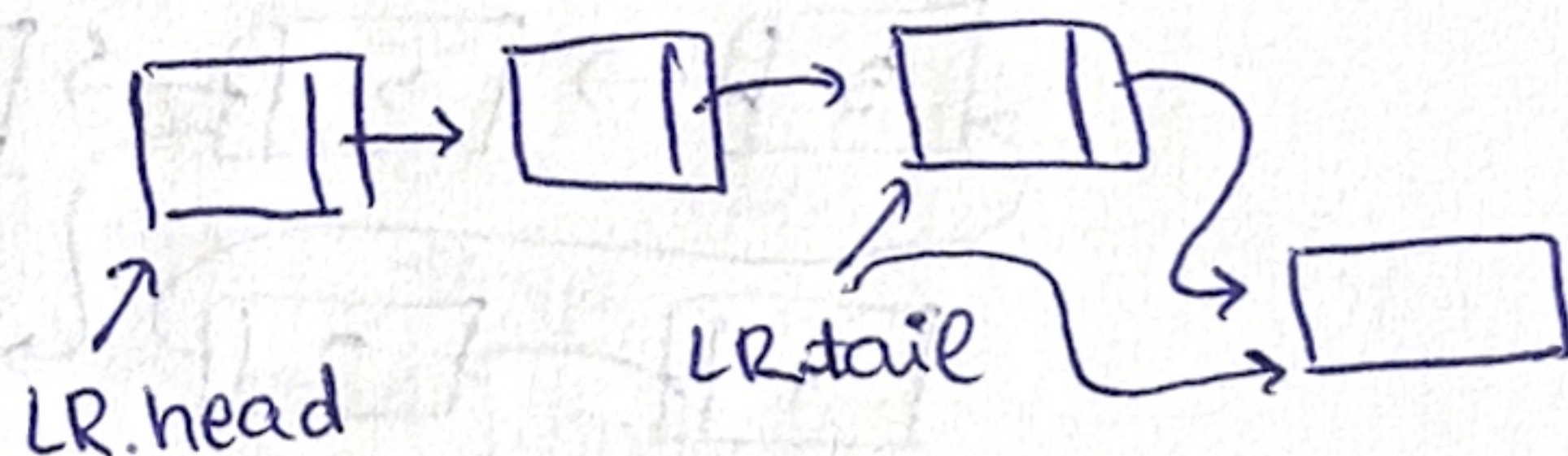
$chosen \leftarrow CL_2$
 $CL_2 \leftarrow [CL_2].next$

$LRtail \leftarrow L_1.head$

$[LRtail].next \leftarrow chosen$

$LRtail \leftarrow chosen$

both head and tail (1 mode)



if $CL_1 \neq Nil$

$[LRtail].next \leftarrow CL_1$

else

$[LRtail].next \leftarrow CL_2$

$[LRtail].next \leftarrow chosen$
 $LRtail \leftarrow chosen$

$LR.R \leftarrow L_1.R$

$L_1.head \leftarrow Nil$

$L_2.head \leftarrow Nil$

3. Radix Sort -

→ variant of lexicographic sort which uses Bucket Sort.

$$\Theta(d \cdot (N+m)) \approx \Theta(m)$$

↓
mr. of numbers that we are sorting
↓
mr. of buckets = how many possible digits. $\Rightarrow 10$.
↓
mr. of digits in the max nr.

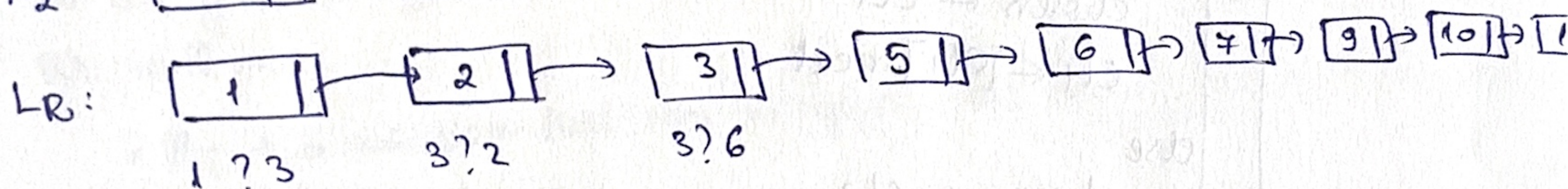
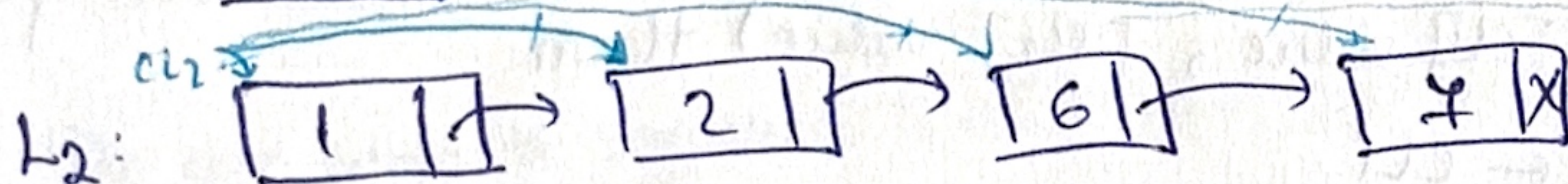
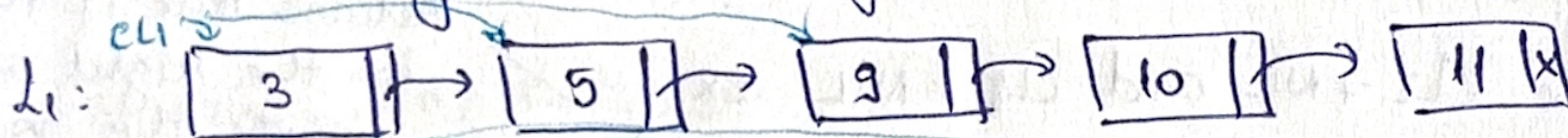
(2, 4, 6) $\Rightarrow$ 246

(0, 3, 5) $\Rightarrow$ 35

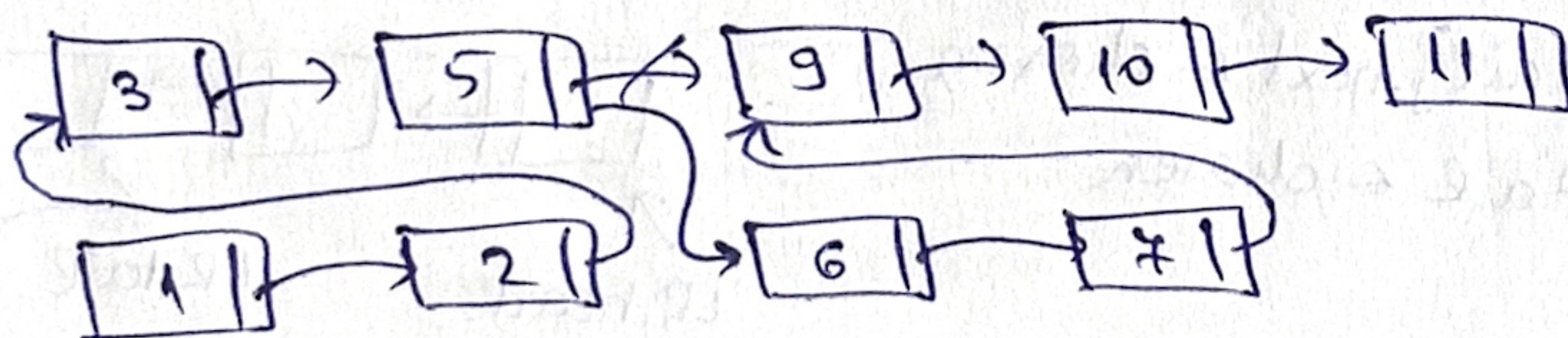
(You need the same length)

→ because it never compares these tuples directly: that's why it's linear.

II. Write a subalgorithm to merge two singly linked list.



(input)
If you don't need L_1 and L_2 (just for merging), you can rearrange the nodes by changing the links.



• Representation:

used with sorted nr.

Node:

value: TComp
next: TNode

List:

head: TNode

R: Relation

(struct)